МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Основная образовательная программа 09.03.02
«Информационные системы и технологии»

Отчет по дисциплине «Алгоритмы и структуры обработки данных»
по лабораторной работе №1

Тема: «Лабораторная работа №1»

Проверил
к.т.н. доцент **Ковалев И.А.**

_____
подпись

Выполнила
студентка группы ИДБ-25-06 **Якунина В.С.**

_____
подпись

Оглавление

ВВЕДЕНИЕ....3
ГЛАВА 1 ПРОСТОЕ ИЗМЕРЕНИЕ ВРЕМЕНИ....4
ГЛАВА 2 ИЗМЕРЕНИЕ ФУНКЦИЙ....5
2.1 Обмен первого и последнего элемента....5
2.2 Итеративный бинарный поиск....7
2.3 Линейный поиск....9
2.4 Проверка, отсортирован ли массив по неубыванию....11
2.5 Сортировка слиянием (merge sort)....13
2.6 Подсчёт инверсий в массиве....15
2.7 Наивный поиск пары элементов с заданной суммой (two-sum)....17
2.8 Поиск тройки с суммой target (с бинарным поиском)....19
2.9 Наивный поиск тройки с суммой target....21
2.10 Подсчёт подмножеств с положительной суммой....23
ЗАКЛЮЧЕНИЕ....25

ВВЕДЕНИЕ

Целью работы является изучение сложности алгоритмов с помощью измерения времени их работы. Работа написана на языке C в операционной системе Windows. Ссылка на github.

ГЛАВА 1 ПРОСТОЕ ИЗМЕРЕНИЕ ВРЕМЕНИ

```
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = func_name(arr, n);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

С помощью данной программы засекается и выводится время работы функции, которую запишут.

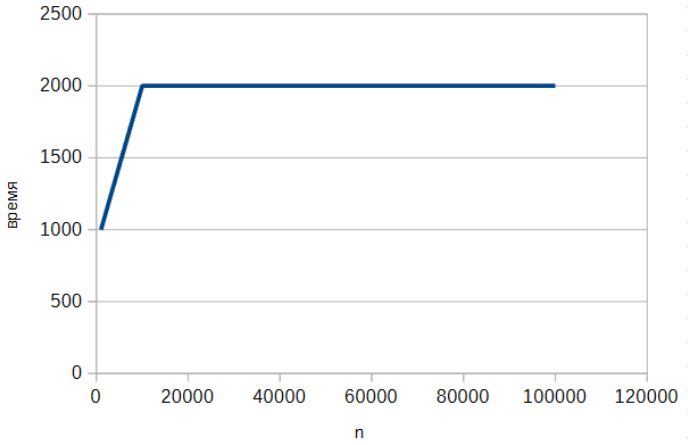
ГЛАВА 2 ИЗМЕРЕНИЕ ФУНКЦИЙ

2.1 Обмен первого и последнего элемента

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

void f02_swap_ends(int *a, int n)
{
    if (n < 2) return;
    int t    = a[0];
    a[0]     = a[n - 1];
    a[n - 1] = t;
}

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    f02_swap_ends(arr, n);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    //printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f02_swap_ends	1000	1000
	10000	2000
	100000	2000



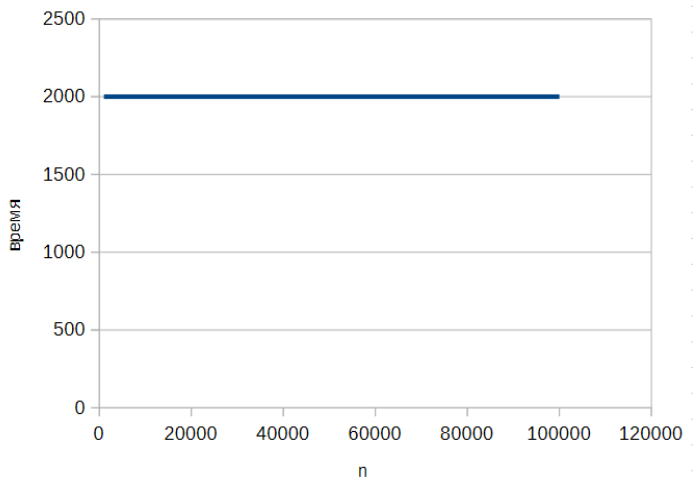
Функция меняет местами первый и последний элементы массива. Сложность: O(1).

2.2 Итеративный бинарный поиск

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

int f03_binary_search(const int *a, int n, int key)
{
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int m = lo + (hi - lo) / 2;
        if      (a[m] == key) return m;
        else if (a[m] <  key) lo = m + 1;
        else                  hi = m - 1;
    }
    return -1;
}

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f03_binary_search(arr, n, 5555);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f03_binary_search	1000	2000
	10000	2000
	100000	2000



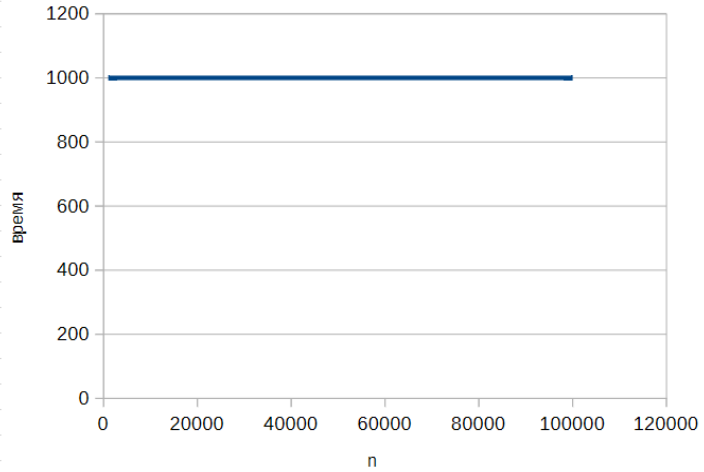
Функция быстро находит поданный в нее элемент. Сложность: O(log n).

2.3 Линейный поиск

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

int f06_linear_search(const int *a, int n, int key)
{
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f06_linear_search(arr, n, 6666);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f12_is_sorted	1000	1000
	10000	1000
	100000	1000



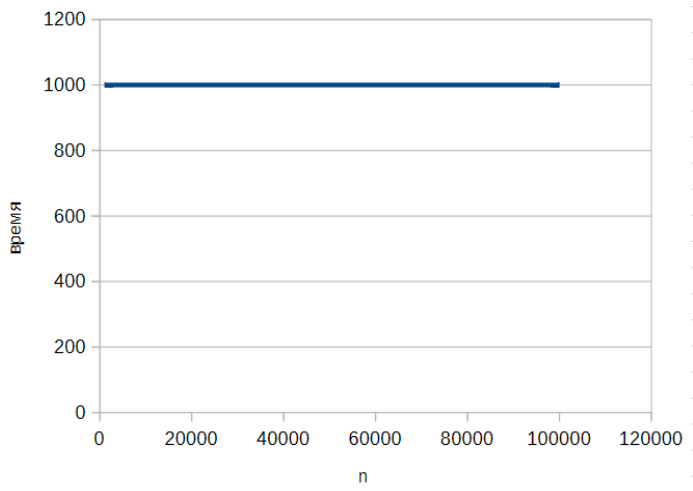
Функция находит поданный в нее элемент. Сложность: O(n).

2.4 Проверка, отсортирован ли массив по неубыванию

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

int f12_is_sorted(const int *a, int n)
{
    for (int i = 1; i < n; i++)
        if (a[i] < a[i - 1]) return 0;
    return 1;
}

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f12_is_sorted(arr, n);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f12_is_sorted	1000	1000
	10000	1000
	100000	1000



Функция проверяет, не отсортирован ли массив по неубыванию. Сложность: O(n).

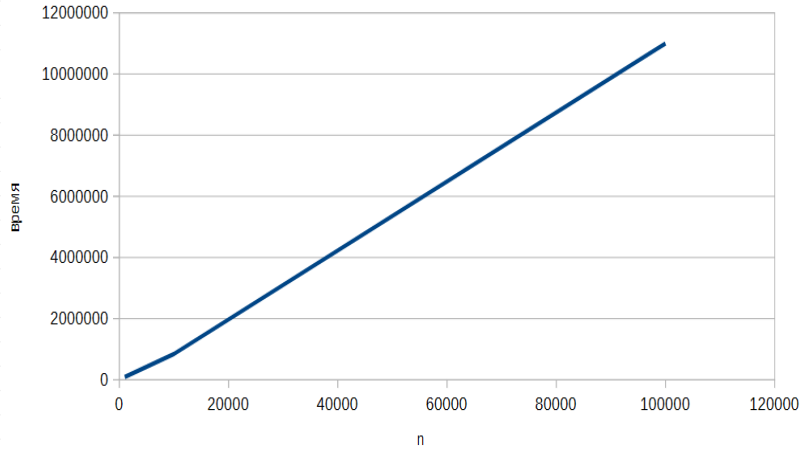
2.5 Сортировка слиянием (merge sort)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

static void ms_merge(int *a, int *buf, int l, int m, int r)
{
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r)
        buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= m) buf[k++] = a[i++];
    while (j <= r) buf[k++] = a[j++];
    memcpy(a + l, buf + l, (size_t)(r - l + 1) * sizeof(int));
}

static void ms_rec(int *a, int *buf, int l, int r)
{
    if (l >= r) return;
    int m = l + (r - l) / 2;
    ms_rec(a, buf, l, m);
    ms_rec(a, buf, m + 1, r);
    ms_merge(a, buf, l, m, r);
}

void f14_merge_sort(int *a, int n)
{
    int *buf = (int *)malloc((size_t)n * sizeof(int));
    ms_rec(a, buf, 0, n - 1);
    free(buf);
}

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    f14_merge_sort(arr, n);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    //printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);
```

```
    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f14_merge_sort	1000	90000
	10000	843000
	100000	11001000



Функция сортирует массив. Сложность: O(n log n).

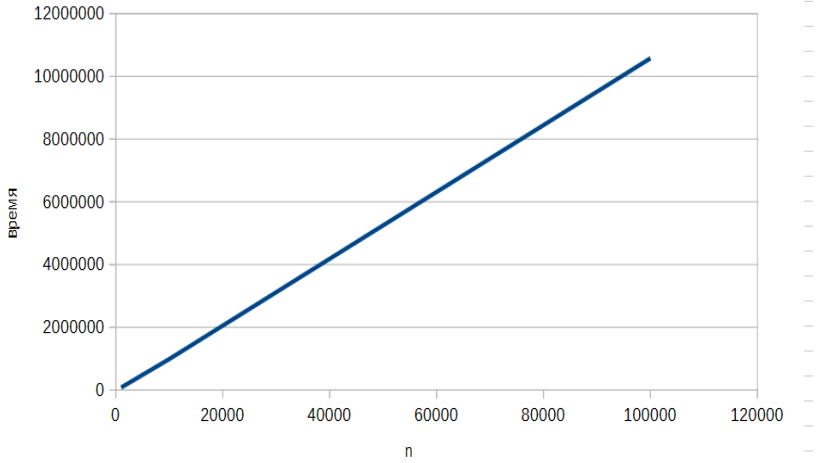
2.6 Подсчёт инверсий в массиве

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

static long long inv_merge(int *a, int *buf, int l, int m, int r)
{
    long long inv = 0;
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) {
            buf[k++] = a[i++];
        } else {
            inv += (m - i + 1); // все оставшиеся в левой половине
            buf[k++] = a[j++];
        }
    }
    while (i <= m) buf[k++] = a[i++];
    while (j <= r) buf[k++] = a[j++];
    memcpy(a + l, buf + l, (size_t)(r - l + 1) * sizeof(int));
    return inv;
}

static long long inv_rec(int *a, int *buf, int l, int r)
{
    if (l >= r) return 0;
    int m = l + (r - l) / 2;
    long long c = 0;
    c += inv_rec(a, buf, l, m);
    c += inv_rec(a, buf, m + 1, r);
    c += inv_merge(a, buf, l, m, r);
    return c;
}

long long f17_count_inversions(const int *a, int n)
{
    int *copy = (int *)malloc((size_t)n * sizeof(int));
    int *buf  = (int *)malloc((size_t)n * sizeof(int));
    memcpy(copy, a, (size_t)n * sizeof(int));
    long long res = inv_rec(copy, buf, 0, n - 1);
    free(copy);
    free(buf);
    return res;
}

int main() {
    int n = 100000;
```

```
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f17_count_inversions(arr, n);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f17_count_inversions	1000	80000
	10000	988000
	100000	10575000



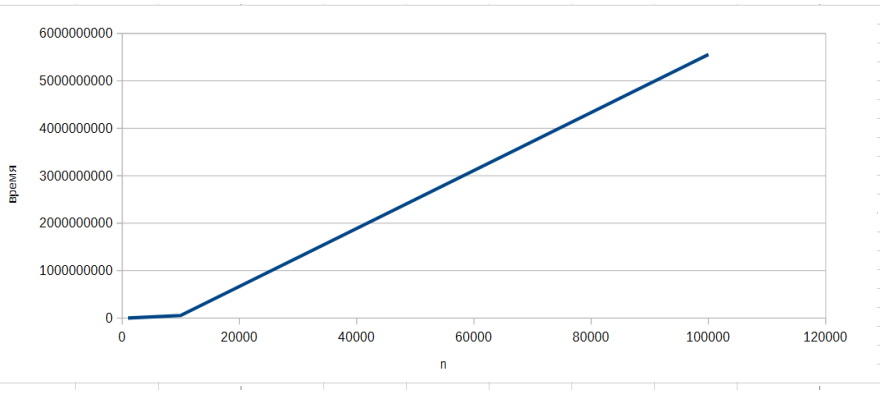
Функция считает количество инверсий в массиве. Сложность: O(n log n).

2.7 Наивный поиск пары элементов с заданной суммой (two-sum)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

int f21_has_pair_sum(const int *a, int n, int target)
{
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] + a[j] == target) return 1;
    return 0;
}

int main() {
    int n = 100000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f21_has_pair_sum(arr, n, 16665);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f21_has_pair_sum	1000	771000
	10000	55681000
	100000	5554206000



Функция ищет в массиве два элемента, которые в сумме дают заданное число. Сложность: $O(n^2)$.

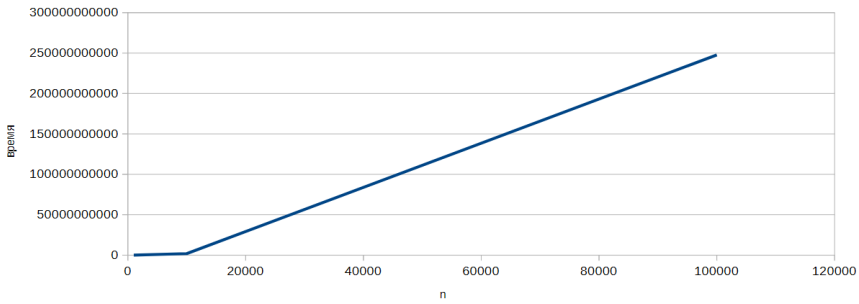
2.8 Поиск тройки с суммой target (с бинарным поиском)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

static void qs_rec(int *a, int lo, int hi)
{
    if (lo >= hi) return;
    int pivot = a[lo + (hi - lo) / 2]; // опорный — средний
элемент
    int i = lo, j = hi;
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++; j--;
        }
    }
    qs_rec(a, lo, j);
    qs_rec(a, i, hi);
}

void f15_quick_sort(int *a, int n)
{
    qs_rec(a, 0, n - 1);
}

int f24_three_sum_bsearch(const int *a, int n, int target)
{
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int need = target - a[i] - a[j];
            int lo = j + 1, hi = n - 1;
            while (lo <= hi) {
                int m = lo + (hi - lo) / 2;
                if      (a[m] == need) return 1;
                else if (a[m] <  need) lo = m + 1;
                else                   hi = m - 1;
            }
        }
    return 0;
}

int main() {
    int n = 1000;
    int *arr = malloc(n * sizeof(int));
```

```
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    f15_quick_sort(arr, n);
    clock_t start = clock();
    long long result = f24_three_sum_bsearch(arr, n, 123400);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f24_three_sum_bsearch	1000	15654000
	10000	1896216000
	100000	247727613000



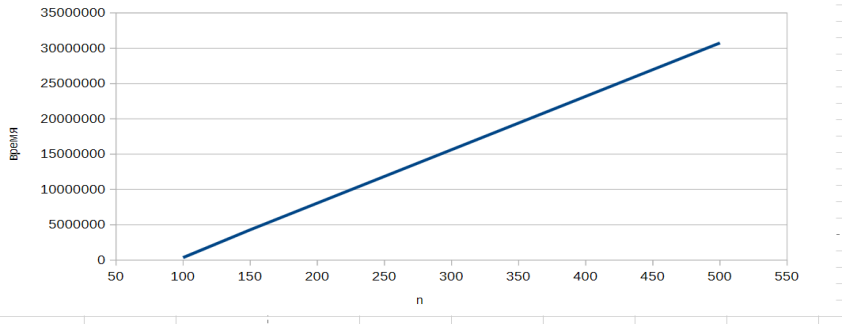
Функция ищет в массиве к паре чисел такое третье, которое в сумме будет давать target, с помощью бинарного поиска. Сложность: $O(n^2 \log n)$.

2.9 Наивный поиск тройки с суммой target

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

int f26_three_sum_naive(const int *a, int n, int target)
{
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            for (int k = j + 1; k < n; k++)
                if (a[i] + a[j] + a[k] == target)
                    return 1;
    return 0;
}

int main() {
    int n = 1000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f26_three_sum_naive(arr, n, 1234567);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f26_three_sum_naive	100	343000
	150	4304000
	500	30745000



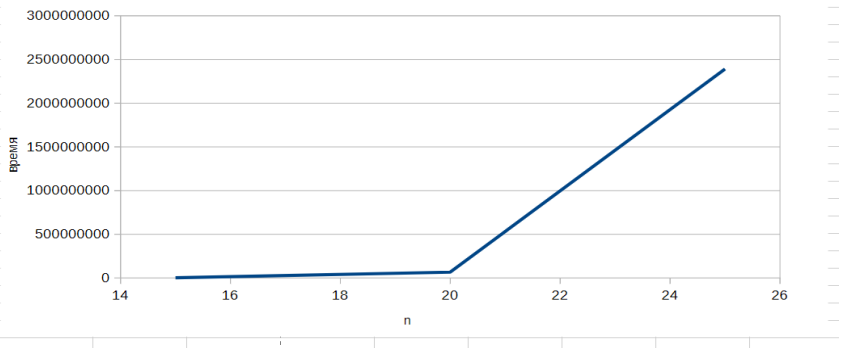
Функция ищет в массиве к паре чисел такое третье, которое в сумме будет давать target. Сложность: $O(n^3)$.

2.10 Подсчёт подмножеств с положительной суммой

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

// 15 20 25

long long f27_positive_subsets(const int *a, int n)
{
    long long cnt = 0;
    long long total = 1LL << n; // 2^n
    for (long long mask = 1; mask < total; mask++) {
        long long s = 0;
        for (int i = 0; i < n; i++)
            if (mask & (1LL << i))
                s += a[i];
        if (s > 0) cnt++;
    }
    return cnt;
}

int main() {
    int n = 1000;
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    long long result = f27_positive_subsets(arr, n);
    clock_t end = clock();

    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Result = %lld\n", result);
    printf("Time = %f seconds\n", time_spent);

    free(arr);
    return 0;
}
```

График зависимости времени выполнения от количества элементов:

Алгоритм	n	Время, нс
f27_positive_subsets	15	2230000
	20	66782000
	25	2390511000



Функция перебирает все 2^n подмножеств с помощью битовой маски. Для каждой маски суммирует выбранные элементы. Сложность: $O(n\ 2^n)$.

ЗАКЛЮЧЕНИЕ

В данной лабораторной работе мы измерили скорость выполнения некоторых алгоритмов, а также рассмотрели зависимость сложности алгоритмов от их скоростей выполнения.